ATIVIDADE - SEMANA 09

Situação de Aprendizagem — Programação Back-end I

Sistema Escolar: POO + Persistência em JSON

INÍCIO	30/09/2026
PROJETO	Cadastro de usuários do Sistema Escolar
CONCEITOS	Herança • Classe abstrata • Polimorfismo
PERSISTÊNCIA	Arquivo usuarios.json
CRUD DA ETAPA	Inserir • Exibir • Excluir
COLABORAÇÃO	PO + Live Share + Git/GitHub

A turma desenvolverá o projeto de forma incremental. Um Product Owner (PO) será sorteado para conduzir a leitura e explicar cada etapa. A digitação será revezada entre os integrantes por meio do Live Share do VS Code, e cada evolução deverá ser registrada no Git/GitHub.

1. Contextualização da Situação de Aprendizagem

A escola precisa de um cadastro simples para organizar usuários do sistema. Existem diferentes perfis — alunos, professores e funcionários — que possuem dados comuns, mas também informações específicas. Nesta primeira evolução, ainda não será utilizado banco de dados: os registros serão persistidos em um arquivo **usuarios.json**.

Desafio

Construir uma aplicação PHP orientada a objetos capaz de inserir, exibir e excluir usuários. A solução deverá demonstrar herança, classe abstrata e polimorfismo e manter os dados mesmo depois de fechar o navegador.

Resultados esperados

- Modelar Usuario como classe abstrata e especializar Aluno, Professor e Funcionario.
- Implementar o método exibirInfo() de maneira polimórfica.
- Ler e gravar uma coleção de registros em usuarios.json.
- Cadastrar novos registros por formulário HTML/PHP.
- Listar registros persistidos no JSON.
- Excluir um registro e regravar o arquivo.
- Trabalhar colaborativamente com Live Share e registrar a evolução no Git/GitHub.

2. Organização da equipe

• **PO:** lê a explicação, faz as perguntas e autoriza a equipe a avançar somente após compreender a etapa.

• **Desenvolvedor:** digita a etapa atual no ambiente compartilhado.

• **Revisor:** acompanha nomes, sintaxe, caminhos e erros.

• **Responsável pelo Git:** executa status, add, commit e push ao término da etapa.

• **Revezamento:** a cada arquivo concluído, troque quem está digitando.

Fluxo

PO explica → equipe discute → aluno digita → equipe testa → corrige → commit → push → troca o responsável.

Estrutura final

```
sistema_escolar/
  index.php
  salvar.php
  excluir.php
  classes/
    Usuario.php
    Aluno.php
    Professor.php
    Funcionario.php
  dados/
    usuarios.json
```

3. Conceitos que serão conectados

Conceito	No projeto
Herança	Aluno, Professor e Funcionario usam extends Usuario.
Classe abstrata	Usuario define a estrutura comum e não pode ser instanciada.
Polimorfismo	Cada classe implementa exibirInfo() de forma própria.
JSON	usuarios.json substitui temporariamente um banco de dados.
CRUD	Nesta etapa: Create (inserir), Read (exibir) e Delete (excluir).

Importante: JSON não é banco de dados. Aqui ele será usado como um mecanismo simples de persistência para tornar visível o processo de serializar, salvar, ler e reconstruir informações.

4. Etapa 1 — Classe Usuario

Usuario concentra nome e e-mail. Como é abstrata, serve de base para os tipos concretos. O método exibirInfo() obriga as classes filhas a definir sua própria apresentação.

```php
<?php
abstract class Usuario
{
    protected string $nome;
    protected string $email;

    public function __construct(string $nome, string $email)
    {
        $this->nome = $nome;
        $this->email = $email;
    }

    public function getNome(): string { return $this->nome; }
    public function getEmail(): string { return $this->email; }

    abstract public function exibirInfo(): string;
}
```

PO — antes de avançar: Por que não podemos criar new Usuario()? O que as classes filhas receberão por herança?

Commit: `feat: cria classe abstrata Usuario`

5. Etapa 2 — Classes filhas

Agora criamos os três tipos concretos. Cada um acrescenta um dado próprio e implementa o mesmo método exibirInfo().

```php
<?php
// Aluno.php
require_once "Usuario.php";

class Aluno extends Usuario {
    private string $matricula;

    public function __construct($nome, $email, $matricula) {
        parent::__construct($nome, $email);
        $this->matricula = $matricula;
    }

    public function exibirInfo(): string {
        return "Aluno: {$this->nome} | Matrícula: {$this->matricula}";
    }
}

// Professor.php e Funcionario.php:
// repetir a mesma ideia usando $disciplina e $cargo
// e implementando exibirInfo().
```

PO — antes de avançar: Por que o método tem o mesmo nome, mas pode produzir respostas diferentes? Complete Professor.php e Funcionario.php antes de continuar.

Commit: `feat: implementa tipos de Usuario`

6. Etapa 3 — Criando usuarios.json

Crie a pasta **dados** e dentro dela o arquivo **usuarios.json**. O sistema começará com um vetor JSON vazio.

```
[]
```

Quando PHP ler esse conteúdo com json_decode(..., true), receberá um array. Quando utilizar json_encode(), fará o caminho inverso e produzirá texto JSON para salvar no arquivo.

PO: peça ao grupo para explicar a diferença entre o array PHP em memória e o texto gravado no arquivo.

Commit: `feat: cria arquivo de persistencia JSON`

7. Etapa 4 — Inserir: formulário em index.php

O formulário será a entrada de dados do CRUD. O campo tipo informa qual perfil está sendo cadastrado. Nesta versão inicial, usaremos um campo extra genérico que representará matrícula, disciplina ou cargo.

```
<form action="salvar.php" method="post">
    <label>Tipo:</label>
    <select name="tipo" required>
        <option value="Aluno">Aluno</option>
        <option value="Professor">Professor</option>
        <option value="Funcionario">Funcionário</option>
    </select>

    <label>Nome:</label>
    <input type="text" name="nome" required>

    <label>E-mail:</label>
    <input type="email" name="email" required>

    <label>Informação específica:</label>
    <input type="text" name="extra" required>

    <button type="submit">Cadastrar</button>
</form>
```

PO — antes de avançar: Qual método HTTP está sendo utilizado? Em qual arquivo os dados serão processados?

Commit: `feat: cria formulario de cadastro`

8. Etapa 5 — salvar.php: gravando no JSON

O arquivo salvar.php recebe os dados enviados pelo formulário. Primeiro lê os registros existentes; depois acrescenta um novo registro e finalmente sobrescreve usuarios.json com a coleção atualizada.

```php
<?php
$arquivo = __DIR__ . "/dados/usuarios.json";

$usuarios = json_decode(
    file_get_contents($arquivo),
    true
);

$novoUsuario = [
    "id" => uniqid(),
    "tipo" => $_POST["tipo"],
    "nome" => $_POST["nome"],
    "email" => $_POST["email"],
    "extra" => $_POST["extra"]
];

$usuarios[] = $novoUsuario;

file_put_contents(
    $arquivo,
    json_encode(
        $usuarios,
        JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE
    )
);

header("Location: index.php");
exit;
```

PO — antes de avançar: Em que momento ocorre o CREATE? Por que primeiro precisamos ler o JSON existente antes de gravar o novo registro?

Commit: `feat: persiste cadastro em JSON`

9. Etapa 6 — Exibir: lendo usuarios.json

Agora index.php fará também o READ. A página carrega o arquivo, converte o JSON em array e percorre os registros.

```php
<?php
$arquivo = __DIR__ . "/dados/usuarios.json";
$usuarios = json_decode(file_get_contents($arquivo), true);
?>

<h2>Usuários cadastrados</h2>

<?php foreach ($usuarios as $usuario): ?>
    <div>
        <strong><?= htmlspecialchars($usuario["nome"]) ?></strong>
        - <?= htmlspecialchars($usuario["tipo"]) ?>
        - <?= htmlspecialchars($usuario["email"]) ?>

        <a href="excluir.php?id=<?= urlencode($usuario["id"]) ?>">
            Excluir
        </a>
    </div>
<?php endforeach; ?>
```

PO — antes de avançar: Qual é o READ desta aplicação? Por que usamos htmlspecialchars() ao mostrar valores recebidos e armazenados?

Commit: `feat: lista usuarios do JSON`

10. Etapa 7 — Excluir: excluir.php

A exclusão recebe o id pela URL, lê o JSON e mantém apenas os registros cujo id seja diferente daquele solicitado. Depois regrava o arquivo.

```php
<?php
$arquivo = __DIR__ . "/dados/usuarios.json";
$id = $_GET["id"] ?? "";

$usuarios = json_decode(
    file_get_contents($arquivo),
    true
);

$usuarios = array_filter(
    $usuarios,
    fn($usuario) => $usuario["id"] !== $id
);

file_put_contents(
    $arquivo,
    json_encode(
        array_values($usuarios),
        JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE
    )
);

header("Location: index.php");
exit;
```

PO — antes de avançar: Qual registro é removido? Para que serve array_filter()? Por que usamos array_values() antes de salvar novamente?

Commit: `feat: exclui usuario do JSON`

11. Integrando POO e JSON

Até aqui o JSON armazenou dados simples. Agora o PO deve conduzir a discussão: **o arquivo JSON guarda objetos PHP?** Não. Ele guarda uma representação textual dos dados. Ao carregar os registros, podemos decidir qual classe deve representar cada item.

```
foreach ($usuarios as $dados) {
    if ($dados["tipo"] === "Aluno") {
        $objeto = new Aluno(
            $dados["nome"],
            $dados["email"],
            $dados["extra"]
        );
    } elseif ($dados["tipo"] === "Professor") {
        $objeto = new Professor(
            $dados["nome"],
            $dados["email"],
            $dados["extra"]
        );
    } else {
        $objeto = new Funcionario(
            $dados["nome"],
            $dados["email"],
            $dados["extra"]
        );
    }

    echo "<p>" . $objeto->exibirInfo() . "</p>";
}
```

Agora a persistência e o polimorfismo se encontram: o JSON informa o tipo; PHP cria o objeto correspondente; e a mesma chamada exibirInfo() produz o comportamento da classe concreta.

Commit: `feat: integra objetos com persistencia JSON`

12. Versionamento e colaboração

```
git status
git add .
git commit -m "feat: cria classe abstrata Usuario"
git push

git add .
git commit -m "feat: implementa cadastro em JSON"
git push

git add .
git commit -m "feat: lista usuarios do JSON"
git push

git add .
git commit -m "feat: exclui usuario do JSON"
git push
```

Não vale deixar todas as alterações para o final. O histórico deve mostrar a evolução real da atividade.

Etapa	Responsável	Commit
Usuario / POO		
Classes filhas		
usuarios.json		
Formulário		
Inserir		
Exibir		
Excluir		
Integração POO + JSON		

13. Testes obrigatórios

- Cadastrar pelo menos 2 alunos, 2 professores e 1 funcionário.
- Atualizar a página e comprovar que os dados permanecem salvos.
- Abrir usuarios.json e relacionar visualmente cada propriedade ao formulário.
- Excluir um registro e verificar se ele também desapareceu do JSON.
- Executar a listagem usando objetos e exibirInfo().
- Tentar instanciar Usuario diretamente e explicar o erro.

14. Desafio da equipe

Crie a classe Coordenador, acrescente a opção ao formulário e faça o sistema inserir, persistir, reconstruir e exibir esse novo tipo sem quebrar os registros anteriores. Depois exclua um Coordenador e confira o JSON.

Desafio extra de raciocínio: o CRUD tradicional possui quatro operações. Nesta atividade implementamos **Create, Read e Delete**. Qual operação ainda falta? Não a implemente nesta etapa; descreva quais arquivos e dados precisariam mudar para acrescentá-la posteriormente.

15. Fechamento da Situação de Aprendizagem

O PO deverá sortear integrantes para responder, sem consultar a apostila:

1. O que é herança e onde ela aparece?
2. Por que Usuario é abstrata?
3. Onde está o polimorfismo?
4. O JSON guarda objetos PHP ou dados?
5. O que json_decode() faz?
6. O que json_encode() faz?
7. Qual trecho corresponde ao Create?
8. Qual trecho corresponde ao Read?
9. Como o Delete foi realizado?
10. Por que cada registro possui um id?
11. Por que o sistema continua com os dados após atualizar a página?
12. Como o Git mostra a evolução do trabalho da equipe?

Critério	Evidência
POO	Aplica herança, abstração e polimorfismo corretamente.
Persistência JSON	Insere, lê e regrava dados sem perder registros.
CRUD da etapa	Inserir, exibir e excluir funcionam.
Código	Arquivos organizados e código compreendido pela equipe.
Colaboração	Há revezamento no Live Share e participação nas decisões.
Git/GitHub	Histórico de commits representa as etapas desenvolvidas.
Explicação	Integrantes conseguem justificar o funcionamento do código.

Entrega: projeto funcional, usuarios.json preservado, CRUD de inserir/exibir/excluir funcionando, integração com as classes, desafio Coordenador e repositório atualizado com commits por etapa.